📚 SETUP GUIDE - Crypto Trading Bot v3.0

Kompletny przewodnik uruchomienia bota tradingowego od zera do produkcji.

📋 Spis treści

1. Wymagania systemowe
2. Paper Trading (Binance Testnet)
3. Live Trading (Binance Mainnet)
4. Konfiguracja LM Studio (LLM)
5. Monitoring i Dashboard
6. Troubleshooting

💻 Wymagania systemowe

Minimalne wymagania

- **OS**: Linux (Ubuntu 20.04+), macOS 12+, Windows 10/11
- **Python**: 3.9 - 3.11
- **RAM**: 4 GB
- **Dysk**: 2 GB wolnego miejsca
- **Internet**: Stabilne połączenie

Zalecane dla LLM (Sentiment Analysis)

- **RAM**: 16 GB (dla modeli 8B)
- **GPU**: RTX 3060+ lub M1/M2 chip (opcjonalne, przyspiesza inference)
- **Dysk**: 10 GB (dla modelu Llama 3 8B)

🧪 Paper Trading (Binance Testnet)

> **Zalecenie**: Zawsze zaczynaj od paper tradingu! Testuj minimum 30 dni przed przejściem na live.

Krok 1: Rejestracja na Binance Testnet

1. **Przejdź do**: https://testnet.binance.vision/
2. **Zaloguj się przez GitHub**:
   - Kliknij “Log In with GitHub”
   - Autoryzuj aplikację Binance Testnet
   - Zostaniesz przekierowany do panelu testnet

```
┌──────────────────────────────────────────┐
│           BINANCE TESTNET                │
│                                          │
│  [🐙 Log In with GitHub]                 │
│                                          │
│  Testnet daje Ci wirtualne środki do     │
│  testowania bez ryzyka prawdziwych pieniędzy│
└──────────────────────────────────────────┘
```

Krok 2: Generowanie API Keys (Testnet)

1. Po zalogowaniu kliknij **“Generate HMAC_SHA256 Key”**
2. Skopiuj i zapisz w bezpiecznym miejscu:
   - `API Key` : `xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx`
   - `Secret Key` : `yyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy`

```
⚠️ UWAGA: Secret Key pokazuje się tylko raz!
Zapisz go od razu, nie będzie możliwości ponownego wyświetlenia.
```

1. **Uzyskaj testowe środki**:
   - Kliknij “Generate” obok BTC, ETH, USDC
   - Każda kryptowaluta ma przycisk do doładowania testowych środków

Krok 3: Instalacja zależności

```
# Przejdź do folderu projektu
cd /home/ubuntu/crypto_trading_bot

# Stwórz wirtualne środowisko
python3 -m venv venv

# Aktywuj środowisko
source venv/bin/activate  # Linux/Mac
# .\venv\Scripts\activate  # Windows

# Zainstaluj zależności
pip install -r requirements.txt
```

Krok 4: Konfiguracja config.yaml

Metoda A - Interaktywna (zalecana):

```
python setup_testnet.py
```

Metoda B - Ręczna:

1. Skopiuj plik konfiguracyjny:

```
cp config.yaml config_testnet.yaml
```

1. Edytuj `config_testnet.yaml` :

```yaml
binance:
  api_key: "YOUR_TESTNET_API_KEY"      # ← Wklej swój klucz
  api_secret: "YOUR_TESTNET_SECRET"    # ← Wklej swój secret
  testnet: true                         # ← MUSI być true!

trading:
  pairs:
    - "BTCUSDC"
    - "ETHUSDC"
    - "SOLUSDC"
    - "ADAUSDC"
    - "AVAXUSDC"
    - "MATICUSDC"
    - "XRPUSDC"
    - "DOGEUSDC"

  initial_capital_per_pair: 5000        # 5000 USDC per para

paper_trading:
  enabled: true

safety:
  force_testnet_only: true              # ← Dodatkowe zabezpieczenie
```

Krok 5: Uruchomienie bota

```bash
# Aktywuj środowisko (jeśli nie aktywne)
source venv/bin/activate

# Uruchom bota w trybie paper trading
python main.py --config config_testnet.yaml

# Lub z dashboard webowym
python main.py --config config_testnet.yaml --dashboard

# Uruchom w tle (background)
nohup python main.py --config config_testnet.yaml --dashboard > bot.log 2>&1 &
```

Krok 6: Monitorowanie

A) Dashboard webowy

```
Otwórz w przeglądarce: http://localhost:5000

Funkcje:
- Aktualne pozycje
- P&L dzisiaj/tydzień/miesiąc
- Wykres equity curve
- Historia trade'ów
- Kelly Criterion stats
- Scale Out progress
```

B) Logi

```
# Podgląd logów na żywo
tail -f logs/trading_bot.log

# Tylko errors
grep "ERROR" logs/trading_bot.log

# Trade'y
grep "TRADE" logs/trading_bot.log
```

C) Discord Notifications

1. Stwórz webhook w Discord:
   - Server Settings → Integrations → Webhooks → New Webhook
   - Skopiuj Webhook URL

2. Dodaj do config:

```
discord:
  enabled: true
  webhook_url: "https://discord.com/api/webhooks/XXXXX/YYYYY"
  bot_name: "🤖 Trading Bot"
```

D) Check Status Script

```
python check_status.py
```

🔴 Live Trading (Binance Mainnet)

> ⚠️ **OSTRZEŻENIE**: Tylko po spełnieniu WSZYSTKICH warunków:
> - Minimum 30 dni profitable paper tradingu
> - Win rate > 55%
> - Sharpe Ratio > 1.0
> - Max Drawdown < 15%
> - Przeczytałeś i rozumiesz ryzyko

Checklist przed startem (OBOWIĄZKOWY)

```
□ Paper trading > 30 dni: ___ dni
□ Win rate > 55%: ____%
□ Sharpe Ratio > 1.0: ____
□ Max Drawdown < 15%: ____%
□ Wszystkie testy przeszły: □ TAK
□ Emergency stop działa: □ TAK
□ Discord notifications działają: □ TAK
□ Backup config.yaml: □ TAK
□ Przeczytałem SAFETY_CHECKLIST.md: □ TAK
□ Zaczynasz z małym kapitałem ($100-500): □ TAK
```

Krok 1: Generowanie API Keys (Mainnet)

1. Zaloguj się na https://www.binance.com
2. Przejdź: Account → API Management
3. Kliknij “Create API”
4. Wybierz **“System generated”** (bezpieczniejsze)
5. Nadaj nazwę: `TradingBot_READONLY` (na start)

```
┌─────────────────────────────────────────┐
│          API KEY PERMISSIONS            │
│                                         │
│  ✅ Enable Reading                      │
│  ✅ Enable Spot & Margin Trading        │
│  ❌ Enable Withdrawals  ← NIGDY!        │
│  ❌ Enable Futures      ← Nie potrzebne │
│                                         │
└─────────────────────────────────────────┘
```

Krok 2: IP Whitelist (KRYTYCZNE!)

```
⚠️ ZAWSZE dodaj IP whitelist! Zabezpiecza przed kradzieżą kluczy.
```

1. W API Settings kliknij “Restrict access to trusted IPs only”
2. Dodaj swój IP serwera:

```
# Sprawdź swój IP
curl ifconfig.me
```

1. Dodaj IP do whitelisty

Krok 3: Konfiguracja config_mainnet.yaml

Metoda A - Interaktywna (zalecana):

```
python setup_mainnet.py
```

Metoda B - Ręczna:

```yaml
binance:
  api_key: "YOUR_MAINNET_API_KEY"
  api_secret: "YOUR_MAINNET_SECRET"
  testnet: false                        # ← Zmień na false!

trading:
  pairs:
    - "BTCUSDC"
    - "ETHUSDC"
    # ... (zacznij od 2-3 par na start)

  initial_capital_per_pair: 100         # ← Zacznij od $100 per para!

paper_trading:
  enabled: false                        # ← Wyłącz paper trading

safety:
  force_testnet_only: false
  max_order_value: 200                  # ← Limit na start

  emergency_stop:
    daily_loss_pct: 5                   # ← Bardziej konserwatywnie
    consecutive_losses: 3
```

Krok 4: Start z minimalnym kapitałem

```
# WAŻNE: Pierwszy start zawsze z --dry-run
python main.py --config config_mainnet.yaml --dry-run

# Jeśli wszystko OK, uruchom normalnie
python main.py --config config_mainnet.yaml --dashboard
```

Krok 5: Monitoring 24/7 (pierwsze 2 tygodnie)

```
KRYTYCZNE: Przez pierwsze 2 tygodnie monitoruj bota aktywnie!

Dzień 1-3:   Sprawdzaj co 2-4 godziny
Dzień 4-7:   Sprawdzaj 3x dziennie
Tydzień 2:   Sprawdzaj 2x dziennie
Potem:       Sprawdzaj rano i wieczorem
```

Użyj:
- Dashboard: http://localhost:5000
- Discord notifications (WŁĄCZ!)
- check_status.py script

Krok 6: Stopniowe zwiększanie kapitału

```
Tydzień 1-2:  $100 per para (testowanie)
Tydzień 3-4:  $250 per para (jeśli profitable)
Miesiąc 2:    $500 per para (jeśli stabilny)
Miesiąc 3+:   Zwiększaj o max 25-50% miesięcznie
```

🧠 Konfiguracja LM Studio

Bot używa lokalnego LLM do analizy sentymentu newsów. Jest to **opcjonalne** ale zalecane.

Krok 1: Instalacja LM Studio

1. Pobierz z: https://lmstudio.ai/
2. Zainstaluj (dostępne na Windows, Mac, Linux)
3. Uruchom aplikację

Krok 2: Pobieranie modelu

1. W LM Studio kliknij **“Discover”**
2. Wyszukaj: `llama-3-8b-instruct`
3. Pobierz wersję **GGUF Q4_K_M** (ok. 4.7 GB)

```
Zalecane modele:
- llama-3-8b-instruct-Q4_K_M.gguf    (4.7 GB, dobra równowaga)
- llama-3-8b-instruct-Q5_K_M.gguf    (5.5 GB, lepsza jakość)
- mistral-7b-instruct-Q4_K_M.gguf    (4.1 GB, szybszy)
```

Krok 3: Uruchomienie serwera

1. Załaduj model (Load)
2. Przejdź do zakładki **“Local Server”**
3. Kliknij **“Start Server”**
4. Domyślny port: **1234**

```
Server URL: http://localhost:1234/v1
```

Krok 4: Testowanie połączenia

```
# Test ręczny
curl http://localhost:1234/v1/models

# Lub przez Pythona
python -c "
import requests
r = requests.get('http://localhost:1234/v1/models')
print('OK!' if r.status_code == 200 else 'ERROR')
"
```

Krok 5: Konfiguracja bota

```yaml
llm:
  enabled: true
  provider: "lm_studio"
  base_url: "http://localhost:1234/v1"
  model: "llama3:8b"
  timeout: 30
  temperature: 0.1
  sentiment_weight: 0.15
```

📊 Monitoring i Dashboard

Dashboard webowy

```
# Uruchom z dashboardem
python main.py --dashboard

# Dostęp
http://localhost:5000
```

Strony:
- `/` - Przegląd (pozycje, P&L, wykresy)
- `/trades` - Historia trade'ów
- `/profitability` - Kelly, Scale Out, Fee stats
- `/api/status` - JSON status (dla integracji)

Discord Notifications

Typy powiadomień:
- 🟢 **Trade Entry**: Otwarcie pozycji
- 🔴 **Trade Exit**: Zamknięcie pozycji (TP/SL/Manual)
- 🎯 **Scale Out**: Częściowe zamknięcie (TP1, TP2, TP3)
- 🚨 **Emergency Stop**: Zatrzymanie bota
- 📊 **Daily Summary**: Podsumowanie dnia (23:00)

🔧 Troubleshooting

Problem: "Cannot connect to Binance"

```
# 1. Sprawdź połączenie internetowe
ping api.binance.com

# 2. Sprawdź czy klucze są poprawne
python -c "
from binance.client import Client
c = Client('YOUR_KEY', 'YOUR_SECRET', testnet=True)
print(c.get_account())
"

# 3. Sprawdź IP whitelist (dla mainnet)
```

Problem: "Insufficient balance"

```
# Testnet: Doładuj środki
# Idź na https://testnet.binance.vision/ → Generate

# Mainnet: Przelej USDC na konto Spot
```

Problem: “Rate limit exceeded”

```yaml
# Zmniejsz częstotliwość w config.yaml
trading:
  check_interval: 60  # Zwiększ do 60 sekund
```

Problem: “LM Studio not responding”

```
# 1. Sprawdź czy serwer działa
curl http://localhost:1234/v1/models

# 2. Sprawdź port w LM Studio

# 3. Restart LM Studio

# 4. Wyłącz LLM jeśli nie potrzebujesz
llm:
  enabled: false
```

Problem: Bot nie otwiera pozycji

```
# 1. Sprawdź logi
tail -f logs/trading_bot.log | grep -E "(DECISION|TRADE|SKIP)"

# 2. Możliwe przyczyny:
# - Min trade score nie osiągnięty (zmniejsz w config)
# - Spread za wysoki
# - Daily limit osiągnięty
# - Pattern effectiveness za niska
```

Jak sprawdzić logi

```
# Wszystkie logi
tail -f logs/trading_bot.log

# Tylko trade'y
grep "TRADE" logs/trading_bot.log

# Tylko errors
grep "ERROR" logs/trading_bot.log

# Ostatnie 100 linii
tail -100 logs/trading_bot.log
```

Jak zrestartować bota

```
# 1. Znajdź proces
ps aux | grep main.py

# 2. Zatrzymaj
kill -SIGTERM <PID>
# lub
pkill -f "main.py"

# 3. Uruchom ponownie
python main.py --config config_testnet.yaml --dashboard
```

Emergency Stop

```
# Natychmiastowe zatrzymanie
pkill -9 -f "main.py"

# Lub przez dashboard
# Kliknij "Emergency Stop" button

# Lub zmień w config
safety:
  emergency_stop:
    manual_stop: true
```

📞 Wsparcie

- **Issues**: Sprawdź logi i troubleshooting powyżej
- **Dokumentacja**: Zobacz README.md i ADVANCED_FEATURES.md
- **Safety**: Przeczytaj SAFETY_CHECKLIST.md przed live trading

---

Pamiętaj: Trading kryptowalut wiąże się z wysokim ryzykiem. Bot jest narzędziem, nie gwarancją zysku. Zawsze traduj odpowiedzialnie i tylko środkami, które możesz stracić.